Do Nothing

The first technique can be easy to overlook—we can simply do nothing. Our code would run inside of the HTTP endpoint and directly enrich the data. It might look like this:

```
def create(conn, params) do
  enriched = enrich_data_via_api!(params)
  data = DataStore.persist!(enriched)
  json(conn, %{id: data.id})
end
```

In this example, we fetch the enriched data and then persist it into the database. This approach has a few benefits. The enriched data is immediately available—so our application could use it right away. Plus, we've written the simplest code possible to solve our problem, so that's an advantage for code cleanliness.

However, we have some significant drawbacks to this approach. If the enrichment API is down, then our API call will fail. If the enrichment API is slow, then users will feel that slowness in our API. Plus, the server has a connection held up while this request processes. (This last drawback is less significant in Elixir because servers have large connection limits compared to Ruby.)

These drawbacks are fairly significant, so let's make the code asynchronous.

Use a Task Process

Elixir provides a module called Task[2] that lets us easily spawn short-lived, task-specific processes. You could do this yourself, of course, but the Task module greatly simplifies it. However, there are shortcomings with Task that make it not a great fit for a job system.

Here's what that could might look like:

```
def create(conn, params) do
  data = DataStore.persist!(params)

  Task.Supervisor.start_child(MyTaskSupervisor, fn ->
    enriched = enrich_data_via_api!(params)
    DataStore.update_enriched!(data, enriched)
  end)

  json(conn, %{id: data.id})
end
```

2. https://hexdocs.pm/elixir/Task.html